

COMPLETE TECHNICAL INTERVIEW MASTER GUIDE

Architecture Block Diagrams, Stored Procedures, Optimization Techniques & Scenario Q&A

Comprehensive Coverage: Spark & Databricks Tuning, Azure & GCP Architectures, Stored Procedures, Incident Automation, and Deep Performance Optimization Pillars (Storage, Compute, Warehouse).

1. DATABRICKS LAKEHOUSE & SPARK ARCHITECTURE

DATABRICKS ARCHITECTURE & OPTIMIZATION

Q1: Walk me through the architecture and code for fixing the 96% Spark bottleneck on the Bybox report.

Architecture Overview: Scanning un-indexed small files across raw storage caused severe IO bottlenecks. The solution involved migrating to Delta Lake with Z-Ordering and Partition Pruning.

[Raw JSON/Parquet] ---> [Delta Bronze (Raw)] ---> [Delta Silver (Z-Ordered)] ---> [Fast Spark Query]

(Over-partitioned) (Auto-Compacted) (OPTIMIZE + ZORDER) (96% Time Saved)

Code Implementation:

```
# 1. PySpark Read with Partition Pruning
df = spark.read.format("delta") \
    .option("spark.databricks.delta.optimizeWrite.enabled", "true") \
    .load("/mnt/bybox_report") \
    .filter("transaction_date >= '2024-01-01' AND account_id = 'ACC_1001'")

# 2. Delta Lake Z-Order Optimization SQL
spark.sql("OPTIMIZE delta.`/mnt/bybox_report` ZORDER BY (account_id, product_id)")
```

AZURE & DATABRICKS INTEGRATION

Q2: How did you structure the competitor price scraping pipeline using ADF, MuleSoft, and Databricks?

Architecture & Data Flow: ADF orchestrates the MuleSoft scraper API, writes raw JSON to ADLS Gen2, triggers Databricks for schema extraction/normalization, and upserts into Azure Synapse.

[ADF Orchestrator] ---> [MuleSoft Scraper] ---> [ADLS Gen2 (Raw JSON)]

[Azure Synapse] <--- [Stored Proc] <--- [Azure Databricks (PySpark)]

Code Snippet: PySpark Databricks Parsing

```
from pyspark.sql.functions import col, explode, current_timestamp

raw_df = spark.read.json("/mnt/adls/raw_competitor_prices/*.json")

parsed_df = raw_df.select(
    col("competitor_id"),
    explode(col("products")).alias("prod")
).select(
    col("competitor_id"),
    col("prod.sku").alias("sku"),
    col("prod.price").alias("competitor_price"),
    current_timestamp().alias("ingestion_timestamp")
)

parsed_df.write.format("delta").mode("append").save("/mnt/adls/silver/competitor_pricing")
```

2. STORED PROCEDURES & WAREHOUSE LOGIC

WAREHOUSE ETL / STORED PROCEDURE

Q3: How do you write an idempotent Merge Stored Procedure for DW / Synapse batch loads?

Data Flow Architecture:



T-SQL Stored Procedure Snippet:

```
CREATE PROCEDURE dbo.MergeCustomerDim
    @BatchId VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    MERGE dbo.DimCustomer AS Target
    USING (
        SELECT CustomerID, CustomerName, Email, UpdatedAt
        FROM dbo.StagingCustomer
        WHERE BatchId = @BatchId
    ) AS Source
    ON Target.CustomerID = Source.CustomerID

    WHEN MATCHED AND Source.UpdatedAt > Target.UpdatedAt THEN
        UPDATE SET
            Target.CustomerName = Source.CustomerName,
            Target.Email = Source.Email,
            Target.UpdatedAt = Source.UpdatedAt

    WHEN NOT MATCHED BY TARGET THEN
        INSERT (CustomerID, CustomerName, Email, UpdatedAt)
        VALUES (Source.CustomerID, Source.CustomerName, Source.Email, Source.UpdatedAt);

    COMMIT TRANSACTION;
END;
```

3. COMPREHENSIVE DATA ENGINEERING OPTIMIZATION PILLARS

OPTIMIZATION TECHNIQUES

Q4: How do you systematically handle tuning across Storage, Compute, and Warehouses?

- **Storage Layer Optimization:**
 - **Small File Problem:** Combine thousands of KB-sized files into 1GB blocks using Delta Compaction (``OPTIMIZE``).
 - **Partition Pruning:** Partition storage folders by low-cardinality keys (e.g., ``Year`/`Month``) to skip unneeded directory scans.
 - **Z-Ordering:** Clustered ordering for multi-column high-cardinality queries (``WHERE account_id = X AND date = Y``).
- **Spark Engine Tuning:**
 - **Data Skew Mitigation:** Enable AQE skew joins (``spark.sql.adaptive.skewJoin.enabled``) or perform *Salting* on skewed join keys.
 - **Broadcast Joins:** Avoid network shuffle overhead by broadcasting small dimension tables using ``broadcast(small_df)``.
- **Cloud DW (BigQuery / Synapse) Optimization:**
 - **Partitioning & Clustering:** Partition tables by timestamp and cluster by up to 4 search columns.
 - **Columnar Projection:** Avoid ``SELECT *`` to reduce disk IO and query costs in columnar execution engines.

4. CLOUD SECURITY & INCIDENT AUTOMATION

GCP SECURITY ARCHITECTURE

Q5: How was secure financial processing implemented on GCP using PGP Encryption?

Architecture Flow:

```
[ Dataproc / Dataflow ] --> [ BigQuery ] --> [ Cloud Storage (GCS) ]
                                     |
[ External Regulators ] <--- [ SFTP ] <--- [ Cloud Function (PGP Encrypt) ]
                                     ^
                                     [ Secret Manager Keys ]
```

Python PGP Encryption Code:

```
import pgpy
from google.cloud import secretmanager

def encrypt_regulatory_report(report_bytes: bytes) -> bytes:
    client = secretmanager.SecretManagerServiceClient()
    key_name = "projects/my-gcp-project/secrets/pgp-public-key/versions/latest"
    pub_key_str = client.access_secret_version(request={"name": key_name}).payload.data.decode("UTF-8")

    key, _ = pgpy.PGPKKey.from_blob(pub_key_str)
    message = pgpy.PGPMessage.new(report_bytes)
    return bytes(key.encrypt(message))
```

AIRFLOW & SERVICENOW AUTOMATION

Q6: How did you automate incident creation to achieve an 80% ticket resolution time reduction?

Automation Flow:

```
[ Airflow DAG Failure ] --> [ Custom Failure Hook ] --> [ Parse Log Exception ]
                                     |
[ L2 On-Call Engineer ] <--- [ Create ServiceNow Ticket ] <-----|
```

Airflow Custom Callback Hook Code:

```
import requests

def servicenow_failure_callback(context):
    exception = str(context.get('exception'))
    task_id = context.get('task_instance').task_id
    dag_id = context.get('task_instance').dag_id

    payload = {
        "short_description": f"Airflow Alert: Task {task_id} failed in {dag_id}",
        "comments": f"Exception Details: {exception}",
        "urgency": "2",
        "assignment_group": "Data_Engineering_Support"
    }

    requests.post("https://instance.service-now.com/api/now/table/incident", json=payload, auth=('user', 'pass'))
```